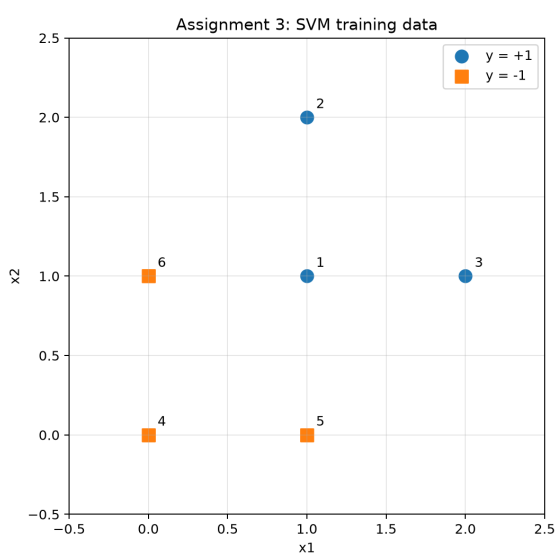
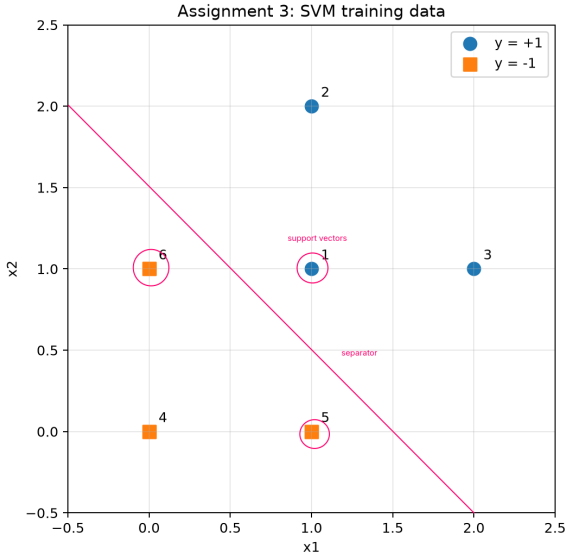
Question 1: Hard Margin Linear SVM

Graphs:





The separator is a line going though (x1,x2) points: (0,1.5) (0.5,1) (1,0.5) (1.5, 0)

x1 + x2 = 1.5.

The points closest to that line (one on each side) are the support vectors:

- point 1 (1,1), y=+1 -> distance |1+1-1.5|/sqrt(2) = 0.354
- point 5 (1,0), y=-1 -> distance |1+0-1.5|/sqrt(2) = 0.354
- point 6 (0,1), y=-1 -> distance |0+1-1.5|/sqrt(2) = 0.354

points 2 (1,2), 3 (2,1), 4 (0,0) are all 1.5/sqrt(2) = 1.06 away, so moving them would not change the boundary. Therefore these points are not support vectors.

We can find the margin and boundary via

*min(1/2 * wlen**2) subject to yn(w*xn+b)>=1 for all n*

the constraint is tight (= 1) exactly at the support vectors, so

y_n (w . x_n + b) = 1 <=> w . x_n + b = y_n (since y_n = +/-1)

- point 1 (1,1): w1 + w2 + b = +1
- point 5 (1,0): w1 + b = -1
- point 6 (0,1): w2 + b = -1

w1 = w2 = 2, b = -3 -> 2*x1 + 2*x2 - 3 = 0 (same as x1 + x2 = 1.5)

The margin is:

margin = (1/w_len) on each side of w

This is the perpendicular distance from the boundary to the closest points on each side:

*1/||w|| = 1/sqrt(2^2 + 2^2) = 1/(2*sqrt(2)) ~= 0.354*

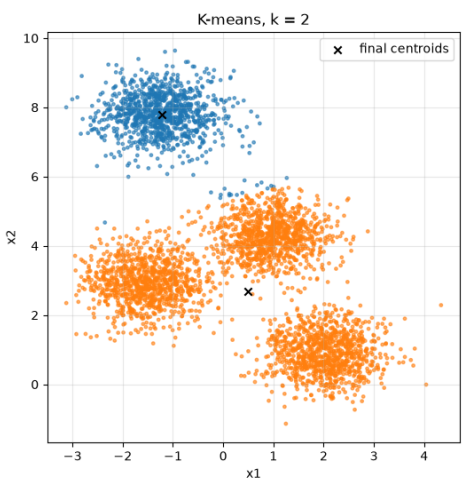
The full width between w.x+b=+1 and w.x+b=-1 is: *2/||w|| = 1/sqrt(2)*

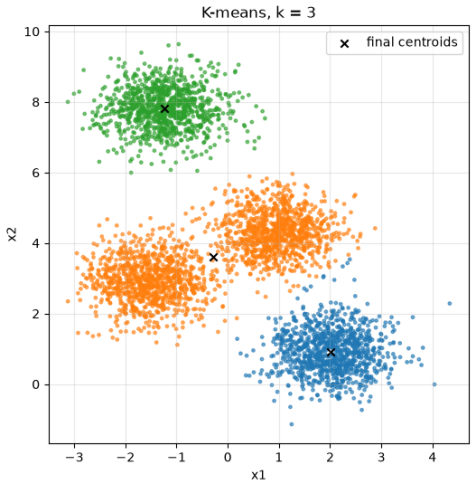
Question 2: K-Means

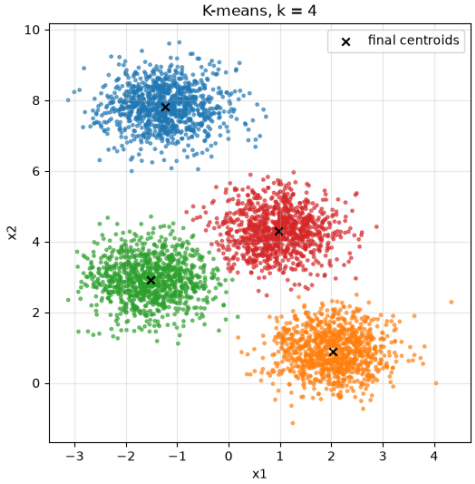
As k increases, it can pass the true labels and instead splits complete groups into individual sections. The smaller k values (k=2,3) group clusters within one larger one, setting k to the true underlying number of clusters (k=4) finds clusters that almost match the true labels, and the larger values (k=6,10) can find clusters but with the extra k numbers, it splits the clusters into smaller ones. This is because the hyper parameter k is always predefined and will create k clusters, even if that is not the true spread of the data. This limitation of always finding a set number of clusters is a weakness of KMeans; as this is a form of unsupervised learning and we would not know the true number of clusters in a real-world setting, choosing too high of a k value will create clusters where there is none.

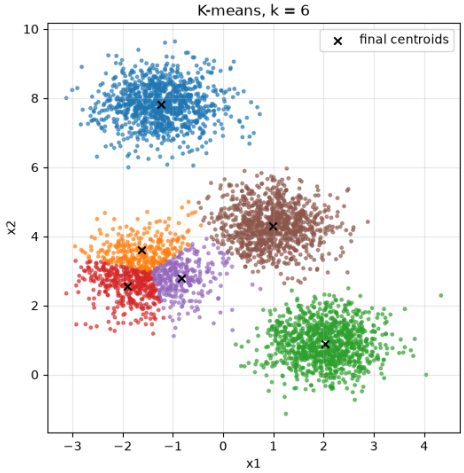
We have to tune the hyperparameter k to determine what might be best. We would do this via BIC or AIC, which use the k that minimizes the loss within clusters while also penalizing a larger k. We need this regularization term because increasing k will always decrease the within-cluster loss.

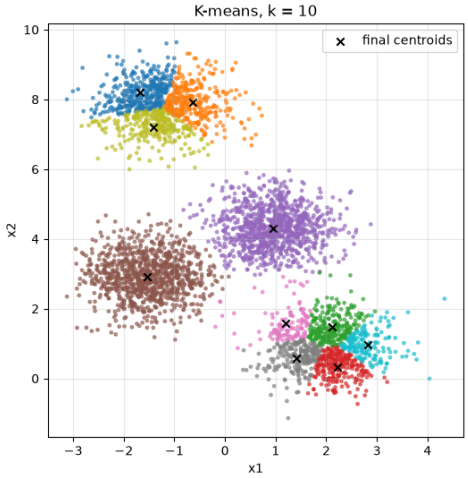
Graphs:

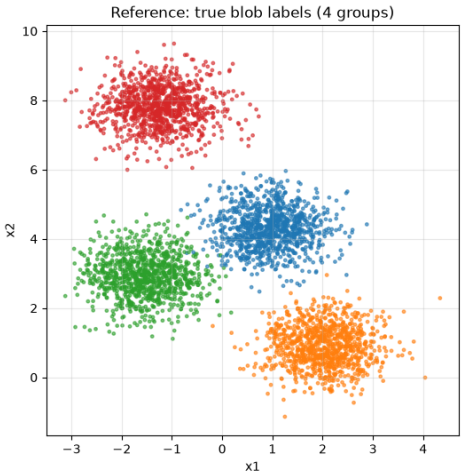





K-means, k = 4
final centroids
x1
x2


K-means, k = 6
final centroids
x1
x2


K-means, k = 10
final centroids
x1
x2


Reference: true blob labels (4 groups)
x1
x2

Question 3: Noisey Label SVM & KNN Repair

Accuracy Comparison:

- Mean baseline accuracy: *0.8280*
- Mean KNN repair accuracy: *0.8644*
- Difference (KNN - baseline): *+0.0364*

We can see that running KNN to denoise labels improves the accuracy compared to the baseline. If a label is flipped in the center of its cluster, the neighbors (which might still retain their original label) help support it and flip it back to its original label. However, for labels that were not flipped but are located on the sides of any cluster might have their true labeled changed during KNN repair, as some neighbors in the same class could have been flipped and are close enough to the other cluster that the neighbors could be in the other cluster; in this case, labels that were not changed during the corruption could be incorrectly switched for points on the edges of the cluster.

This means that while the labels in the center of the cluster are likely to be corrected, the ones on the sides might be incorrectly changed. Since most data is located in the center of the cluster, we see an overall improvement in accuracy. This improvement was seen in al trials.